

Part B – Weekly Capstone Assignment Template

git repo: <https://github.com/michael-d-abraham/Machine-learning-Standby-Predictor>

This template is used for **each of the 14 Part B capstone submissions** throughout the semester. The structure remains the same every week so that you can focus on *thinking and judgment*, not guessing expectations.

Your goal is not to achieve the best performance, but to reason carefully about how this week’s machine learning technique applies (or does not apply) to your project.

1. Project Context (Brief)

Provide the same short context each week so graders can orient quickly.

- **Project Title:** Content-Based Music Recommender (Spotify Audio Features + Lyrics/Topics)
 - **Data Modality:** tabular
 - **Task Type:** recommendation (content-based similarity search)
 - **One-Sentence Goal:** Given a song, recommend other songs that sound similar using the song’s features.
-

2. This Week’s Technique and Its Assumptions

- **Technique / Model Family Covered This Week:** Practical deep learning systems (end-to-end training pipeline, environment/reproducibility, batched optimization, and observability—not architectural novelty)
- **Key Assumptions of This Technique:**
 - A stable **software environment** (framework version, device choice) is part of the result, not an afterthought.
 - **Train/validation discipline** matters: the same leakage-safe splits and preprocessing rules as the rest of the capstone should carry into the DL loop.
 - **Monitoring** (loss curves, simple metrics) is how you detect overfitting, divergence, or implementation bugs—especially before scaling compute.
 - **Tabular, medium-scale** data on one machine often does not need distributed training or serving infrastructure; many “systems” lessons still apply at laptop scope (seeds, logging, wall-clock).

Fit Assessment (required):

Practical deep learning workflows are a **partial fit** for my project: they apply whenever I train a neural model, but my capstone data

are **hand-engineered numeric features** (~23 columns, ~28k rows) with an **artist-group split**. The main engineering constraints are **reproducibility** and **honest generalization**, not building a large distributed stack. A small supervised proxy (binary energy) is enough to exercise the pipeline without pretending the recommender itself is solved by one classifier.

3. Representation or Proxy Used

Describe how your data was represented so that this week's technique could be applied.

Examples include:

- Hand-engineered features
 - Summary statistics
 - Frozen embeddings
 - Dimensionality reduction
 - A proxy task
 - **Representation or Proxy Chosen:** The same **23 numeric song-level features** used elsewhere (topic proportions + audio descriptors + age). Excluded: `Unnamed: 0`, `artist_name`, `track_name`, `lyrics`, `genre`, `topic`, `release_date`. **Target:** continuous **energy** binarized to high/low using the **training-set median only** (no test leakage), matching `main.split_data`. Preprocessing: `SimpleImputer(median)` and `StandardScaler` fit **on training rows only**, then applied to val and test.
 - **Why this representation was reasonable for this week:** This week's focus is the **training system** (loop, batches, device, logging). A fixed tabular tensor after sklearn preprocessing mirrors how many production pipelines wrap classical feature engineering before a neural head. Raw lyrics/audio end-to-end modeling would be a different (much larger) system scope.
-

4. What Was Attempted

Be concrete and scoped. Do not list everything you *could* have done.

- What you implemented this week Implemented `assignment12_dl_systems.py`: loads `tcc_ceds_music.csv`, uses `main.inspect_dataset` and `main.split_data` for the **artist-based 60/20/20 split** and **median-derived binary**

label, fits **imputer + scaler on train only**, then trains a **small PyTorch MLP** (23-64-32-1 logits) with **Adam**, **BCE-with-logits**, **batch size 256**, **40 epochs**, fixed **seed 42**. Each epoch logs **mean train loss** and **validation loss** plus **validation ROC-AUC**. Run metadata and tail of metrics are written to **assignment12_dl_run.json**. Device is **CPU** on my run (GPU not required for this scope).

- What you intentionally did *not* attempt and why Did **not** add distributed training, hyperparameter search, model serving, or cloud experiment tracking—to keep the assignment scoped to **single-machine** “systems hygiene.” Did **not** train on raw text or waveforms; the capstone representation stays **tabular** for comparability with prior assignments.
- Any constraints encountered (data, labels, compute, time) Full recommender **ground truth** is still absent; the energy task remains a **proxy** for reasoning about features and generalization. Training is **fast on CPU** for this net (~seconds); the bottleneck for a real DL system here would be **data/modeling choices**, not cluster scale.

5. Results or Observations

You may include metrics **if applicable**, but qualitative observations are also valid.

Examples:

- Evaluation metrics
- Training behavior or convergence issues
- Error patterns
- Unexpected behaviors

Data split (artist-group): Train **16,615** / Val **5,616** / Test **6,141** rows; **23** features after exclusions.

Environment / run config (from assignment12_dl_run.json): PyTorch **2.11.0**, **CPU**, seed **42**, batch **256**, lr **0.001**, hidden (**64**, **32**), **40** epochs. Wall-clock for the full train+eval loop about **3.5 s** on this machine.

Training behavior: Train loss decreased steadily (about **0.56** to **0.28**). Validation ROC-AUC rose quickly then plateaued (about **0.918** epoch 1 to ~**0.941-0.943** mid-run). Validation loss **bottomed early** then **crept upward** while train loss kept falling, which is a typical **mild overfitting** signal; not tuned aggressively per assignment intent.

Held-out test: ROC-AUC about **0.939** (single run, not competitive benchmarking).

6. Interpretation and Judgment

This section matters most.

Reflect on:

- Why the method behaved as it did
- Which assumptions held or failed
- What this reveals about your data or problem framing

Why the method behaved as it did:

The network fit the proxy task quickly because the inputs are **dense numeric summaries** already aligned with **energy**. The **artist split** still enforces non-trivial generalization (no shared artists across splits). The **train/val loss gap** suggests extra capacity and epochs mainly **memorize train noise** rather than unlock new “structure”—consistent with a **partial** case for deep models on this **fixed feature matrix**.

Systems assumptions that held:

Reproducibility (seed), **explicit device**, **batched training**, and **JSON logging** made runs **inspectable** and **comparable**—this is the core “practical systems” takeaway for a small project.

What this reveals:

Practical DL **infrastructure** (loop + metrics + artifacts) is **portable** even when the **model** is tiny. For this capstone, the harder constraint remains **evaluation framing** (proxy labels, artist leakage) rather than **framework choice**. End-to-end audio or text DL would shift the systems problem toward **data volume, I/O, and labeling**—not applicable as a full build **at this stage** without expanding scope.

This suggests that while a neural net can match strong tabular baselines here, the **marginal value** of deep learning is often in **pipeline discipline** and **future representation learning**, not a deeper MLP on the same 23 columns.

7. Forward-Looking Adjustment

Answer **one** of the following:

- What will you keep, change, or discard before the next assignment?
- What would you try next if data or resources were not constrained?

I’ll **keep** the **artist-group split** and **median-only thresholding** for any future neural experiments. I’ll **keep** small JSON run logs for reproducibility.

Limitations: One short run; no early stopping or tuned regularization—intentionally minimal.

Next steps: If expanding later: **early stopping**, weight decay, or a **learned embedding** from audio/text—only if the task moves beyond fixed tabular features. For recommendation itself, continue to emphasize **similarity/ranking** rather than a single proxy classifier score.

8. Mismatch Acknowledgment (Complete Only If Applicable)

If this week's technique was a poor fit, explain:

- Why it does not align with your project
- Evidence supporting that conclusion
- What value this attempt still provided

Not a full mismatch. **Distributed training, production serving, and large-scale experiment platforms** are **not applicable for this project at this stage**—dataset size and modality do not justify that machinery yet. The value of this week is practicing **disciplined training loops** and **honest fit assessment** on real capstone splits; the small MLP is a **vehicle** for that systems thinking, not a claim of SOTA recommendation.

Submission Notes

- Written submission format: **Markdown or PDF**
- Code or notebooks: `assignment12_dl_systems.py` (run produces `assignment12_dl_run.json`)
- Performance is **not** graded competitively
- Clear reasoning and honest reflection matter more than results